Bug Severity Classification with Fine-Tuned Open LLMs: A Paired Comparison with TF-IDF and CodeBERT Baselines on Eclipse Reports

Asset Kanatov[a], Talgar Bayan[a] and Askar Boranbayev[a,*]

[a]*Department of Computer Science, Nazarbayev University, Astana, Kazakhstan*

ARTICLE INFO

Keywords:
Bug severity prediction
Bug triage
Large language models
QLoRA
Parameter-efficient fine-tuning
CodeBERT
Mining software repositories

ABSTRACT

Open large language models (LLMs) small enough to run on a laptop are an obvious candidate for automatic bug severity prediction. We measure what they add over cheaper classifiers on one task: assigning one of four Bugzilla severity levels (Critical, Major, Minor, Trivial) to Eclipse bug reports. From a public dump of 361,006 unique Eclipse reports we draw a class-balanced sample of 20,000 rows and compare eleven configurations of nine model types: six TF-IDF classifiers, a fine-tuned CodeBERT encoder, and Llama 3 8B and Qwen 2.5 7B, each run zero-shot and after QLoRA fine-tuning. All eleven are scored on the same 2,000 held-out reports, with bootstrap intervals and McNemar tests. The zero-shot runs reach macro-F1 0.305 and 0.321, below every TF-IDF model (0.430–0.489). The fine-tuned runs reach 0.526 and 0.485, but the zero-shot and fine-tuned setups also differ in checkpoint, serving runtime and decoding, so this gap cannot be credited to fine-tuning alone. Fine-tuned Llama 3 shows no detectable difference from CodeBERT (0.518; difference +0.007, 95% CI −0.017 to +0.032), and fine-tuned Qwen 2.5 is below it on macro-F1. The models differ more by class than in aggregate: fine-tuned Llama 3 finds 36% of Critical reports, Qwen 2.5 finds 69% but only 21% of Minor ones. The dump repeats 60,000 rows, and 8% of the test reports duplicate a training report; removing them lowers TF-IDF scores by up to 0.031 and leaves the LLMs almost unchanged. We release split identifiers, per-report predictions and the analysis script, together with CyberTriage, a tool that serves the fine-tuned classifier locally.

1. Introduction

When a reporter files a bug in Bugzilla, they pick a severity from a fixed list that runs from *blocker* to *trivial*, and triagers use that field to decide what gets looked at first. A wrong value costs most when the defect is a security or privacy weakness, the kind of problem that secure development lifecycles try to catch and rank early (Bayan and Banach, 2023; Bayan, Yazici and Banach, 2025b). The field is often unreliable. In the Eclipse collection we study, two thirds of all reports are labelled *normal*, the default value, and Tian, Ali, Lo and Hassan (2016) found that about half of the duplicate reports in Eclipse, Mozilla and OpenOffice carry different severities even though they describe the same problem. Predicting severity from the report text has therefore been studied for well over a decade, starting with rule learners and Naive Bayes over word features (Menzies and Marcus, 2008; Lamkanfi, Demeyer, Giger and Goethals, 2010) and moving through deep networks (Ramay, Umer, Yin, Zhu and Illahi, 2019; Dao and Yang, 2021) to pre-trained encoders (Ali, Xia, Umer and Osman, 2024; Agarwal and Madhavan, 2026) and, recently, generative LLMs (Silva de Oliveira, 2024; Arshad, Riaz, Fatima and Yasin, 2024; Ali, Khan, Xia, Tian, Sajid and Alsuhaibani, 2025).

Open models with 7–8 billion parameters are a natural next step. QLoRA lets them be adapted on a single GPU (Dettmers, Pagnoni, Holtzman and Zettlemoyer, 2023), and once quantised they run on a laptop, so a team could keep its reports in-house. What is less clear is whether they beat a fine-tuned encoder that is sixty times smaller. Published comparisons rarely settle this, because they differ in data, label scheme, split and metric, and few of them score every model on the same reports with a measure of uncertainty.

*Corresponding author
asset.kanatov@nu.edu.kz (A. Kanatov); talgar.bayan@nu.edu.kz (T. Bayan); askar.boranbayev@nu.edu.kz (A. Boranbayev)

This paper compares eleven configurations of nine model types on one task and one split: TF-IDF classifiers, a fine-tuned CodeBERT encoder (Feng, Guo, Tang, Duan, Feng, Gong, Shou, Qin, Liu, Jiang and Zhou, 2020), and two open LLMs, Llama 3 8B (Grattafiori et al., 2024) and Qwen 2.5 7B (Qwen Team, 2024), used both zero-shot and after QLoRA fine-tuning. All are scored on the same 2,000 test reports. The work grew out of a master's thesis. The LLM scores reported in that thesis, and those shown by the evaluation screen of the original CyberTriage code, were computed by a scoring routine that added synthetic correct predictions, 25% as many as the real ones, before computing the metrics. For this paper we recomputed every number with one script from the raw data dump, the real per-report predictions and the training logs; these numbers supersede the thesis values, and we report only what the records support. The code released with this paper no longer uses that routine and keeps it only as a record. As in our earlier data releases, which publish the raw outputs and aggregated metrics of every run (Bayan, Mukhambetiyar, Boranbayev and Yazici, 2026b,a), we release the split identifiers, the per-report predictions and the analysis script. Three questions guide the analysis.

RQ1. On identical test reports, how do QLoRA-fine-tuned 7–8B LLMs compare with TF-IDF classifiers and a fine-tuned CodeBERT encoder?

RQ2. What changes when the same two model families are fine-tuned instead of prompted zero-shot, overall and for each severity level?

RQ3. How sensitive are these comparisons to two properties of the data: reports duplicated across the split, and the balanced class distribution?

The paper makes four contributions:

1. A paired comparison of eleven configurations on the same 2,000 reports, with bootstrap intervals, McNemar tests and Holm correction, released with split identifiers and per-report predictions.
2. An audit of the public Hugging Face dump used here, showing that it holds Eclipse reports only, repeats 60,000 rows, has fewer unique Trivial reports than the sampling asked for, and lets 8% of the test reports duplicate a training report, together with the measured effect of each issue.
3. Measurements of the two fine-tuned LLMs in this setting: far above the zero-shot runs, though in a setup that differs in more than training; no detectable difference from CodeBERT at about 27 times its per-report latency; and different per-class trade-offs between the two.
4. CyberTriage, an open web tool that serves the fine-tuned classifier on a laptop and adds a rationale written by a second LLM, which cannot change the stored label.

Section 2 reviews severity prediction. Section 3 describes the data and what the audit found, and Section 4 the models and evaluation protocol. Section 5 answers the three questions, Section 6 describes CyberTriage, Section 7 discusses the findings and threats to validity, and Section 8 concludes.

2. Related work

2.1. Severity prediction from report text

Menzies and Marcus (2008) were among the first to predict severity automatically, applying the Ripper rule learner to terms selected with TF-IDF and information gain from NASA defect reports with five severity levels. Lamkanfi et al. (2010) framed the Bugzilla version of the task as a binary one, grouping *blocker*, *critical* and *major* as severe and *minor* and *trivial* as non-severe, and trained Naive Bayes on three open-source projects; a follow-up comparison on Eclipse and GNOME found multinomial Naive Bayes ahead of plain Naive Bayes, k-nearest neighbours and support vector machines (Lamkanfi, Demeyer, Soetens and Verdonck, 2011). Tian, Lo and Sun (2012) kept the five Bugzilla levels and predicted them with a BM25-based nearest-neighbour method whose similarity weights were learned from duplicate reports, and Yang, Zhang and Lee (2014) combined topic models with several report features to find similar historical reports for severity prediction. Later work replaced hand-built features with neural networks: Ramay et al. (2019) fed text vectors and an emotion score to a deep network, and Dao and Yang (2021) combined content, sentiment, quality and reporter features in a convolutional network evaluated on Eclipse and Mozilla.

Two points from this work matter here. Most studies drop the *normal* label, because it is the Bugzilla default and many reporters never change it (Silva de Oliveira, 2024), and the label itself is noisy: Tian et al. (2016) found inconsistent severities for about 51% of duplicate reports and argued that agreement with the recorded label is an imperfect yardstick.

Table 1
Selected severity-prediction studies from Section 2. Label schemes and data differ, so reported scores cannot be compared across rows. A dash marks information we could not confirm from the published abstract or record.

Study	Data	Labels	Models
Menzies and Marcus (2008)	NASA PITS	five NASA levels	Ripper rule learner
Lamkanfi et al. (2010)	three open-source projects	severe vs. non-severe	Naive Bayes
Lamkanfi et al. (2011)	Eclipse, GNOME	severe vs. non-severe	NB, multinomial NB, k-NN, SVM
Tian et al. (2012)	three open-source projects	blocker to trivial	BM25 nearest neighbour
Dao and Yang (2021)	Eclipse, Mozilla	—	CNN with multi-aspect features
Silva de Oliveira (2024)	Eclipse, Mozilla	severe vs. non-severe	Llama 2 (prompted, fine-tuned)
Agarwal and Madhavan (2026)	—	—	GraphCodeBERT and other code models, LoRA
Ali et al. (2025)	mobile-app reports	—	fine-tuned GPT-2 with retrieval
This study	Eclipse	four levels	TF-IDF, CodeBERT, Llama 3, Qwen 2.5

2.2. Pre-trained encoders and generative models

Pre-trained transformers entered the task as encoders. Ali et al. (2024) fine-tuned BERT together with a sentiment signal on mobile-application reports, and Agarwal and Madhavan (2026) compared fine-tuned code models for severity prediction, including GraphCodeBERT trained in full and with LoRA. Generative models followed. SevPredict fine-tunes GPT-2 and reports gains over a BERT-based predictor (Arshad et al., 2024); RAG-GPT-SBR adds retrieved historical reports and metadata to a fine-tuned GPT-2 with a classification head (Ali et al., 2025); and Silva de Oliveira (2024) prompted and fine-tuned Llama 2 for the severe versus non-severe split on Eclipse and Mozilla. Table 1 summarises these studies. Their scores are not comparable with each other or with ours, since data, label schemes and splits differ.

Beyond severity, Hou, Zhao, Liu, Yang, Wang, Li, Luo, Lo, Grundy and Wang (2024) survey LLM use across software engineering, and Kiashemshaki, Khosravani, Hosseinpour and Akhavan (2025) apply instruction-tuned LLMs with constrained decoding to developer assignment, a different triage task. LoRA (Hu, Shen, Wallis, Allen-Zhu, Li, Wang, Wang and Chen, 2022) trains low-rank updates next to frozen weights, and QLoRA (Dettmers et al., 2023) keeps the frozen weights in 4-bit NormalFloat (NF4) so that a 7–8B model can be tuned on one GPU. We use QLoRA as an off-the-shelf method; it is not a contribution of this paper.

3. Data

3.1. Source and provenance

We use the Hugging Face dataset `AndressaStefany/bug-reports`[1], a CSV export with six columns: an internal record identifier, the Bugzilla `bug_id`, the free-text `description`, `bug_severity`, `bug_status` and `resolution`. The dataset has no card, so we identified it from its contents. The file holds 421,006 rows, but the last 60,000 rows are exact copies of earlier ones (bug IDs 1 to 62,767), leaving 361,006 unique reports with bug IDs up to 424,785. The repeats come from the dataset repository itself: it stores two CSV files, `eclipse_clear.csv` with 361,006 rows and `my_dataset.csv` with 60,000 rows that copy the first 60,000 of the former, and the standard loader joins both into one split. That count and the per-label counts in Table 2 match exactly the Eclipse collection described by Silva de Oliveira (2024), who also used a separate Mozilla collection of 768,335 reports that is not part of this dump. The resolution values include `NOT_ECLIPSE`, and 38% of the unique descriptions mention Eclipse by name. We therefore treat the data as Eclipse-only.

Each report is represented by its description alone; the dump has no summary line. Every report in it is resolved, closed or verified, but status and resolution were not given to any model. Some descriptions, mostly from the earliest years, carry text added after filing, such as a `NOTES:` section or dated entries signed with initials; 201 of the 20,000 sampled descriptions (1.0%) contain such markers. The severity is the value

[1] https://huggingface.co/datasets/AndressaStefany/bug-reports

Table 2
Labels in the Hugging Face dump and how the experimental sample was drawn. "Rows" counts every row in the file; "Unique reports" drops the 60,000 repeated rows. The pool is what the sampler drew from; it still contains repeated rows.

Label in the dump	Rows	Unique reports	Mapped class	Pool rows	Sampled (train / test)
`blocker`	6,972	6,223	Critical	22,127	4,000 / 1,000
`critical`	15,255	12,845			
`major`	40,743	35,367	Major	40,530	4,000 / 1,000
`minor`	15,539	13,438	Minor	15,461	4,000 / 1,000
`trivial`	5,411	4,815	Trivial	5,370	4,000 / 1,000
`normal`	282,829	242,819	excluded	—	—
`enhancement`	54,257	45,499	excluded	—	—
Total	421,006	361,006		83,488	16,000 / 4,000

of the field when the data were collected; the dump keeps no history, so we cannot tell initial labels from revised ones.

3.2. Labels and exclusions

Table 2 lists every label in the dump. Two of them were excluded: *normal*, which covers 67.3% of unique reports and is the Bugzilla default, and *enhancement*, which marks feature requests (12.6%). The remaining five were mapped to four classes by merging *blocker* and *critical* into Critical. After the mapping, 432 rows with empty descriptions were dropped, leaving a pool of 83,488 rows (72,337 unique reports). The pool is far from balanced and is dominated by the more severe classes: among unique reports, Major makes up 48.6%, Critical 26.3%, Minor 18.5% and Trivial 6.6%.

3.3. Sampling, split and what the split contains

The preparation script (`dataset/1_prep_balance_split_csv.py`) drew 5,000 rows per class without replacement from the pool with `random_state=42` and split the 20,000 rows 80/20, stratified by class, into 16,000 training and 4,000 test reports. Rerunning the script on the dump reproduces both files exactly, which lets us attach a bug ID to every row; the identifiers are released with this paper.

Sampling from rows rather than unique reports has two consequences. First, the pool holds only 4,778 unique Trivial reports, fewer than the 5,000 requested, so repeated rows were bound to enter the sample; 962 bug IDs appear twice in the 20,000 rows, 519 of them Trivial, so the sample holds 19,038 distinct reports. Second, repeats can land on both sides of the split: 319 test reports (8.0%) have a description identical to a training report, 175 of them Trivial. Balancing before splitting does not by itself prevent this kind of leakage. Section 5.3 measures its effect. The text itself also shows some label noise: among the unique pool reports, 41 descriptions are filed under more than one bug ID with conflicting severities (243 reports in all). We did not inspect a sample of reports manually, so we make no claim about the overall noise rate.

The sample mixes outcomes. Only 64.4% of the 20,000 reports were resolved as `FIXED`; the rest were closed as duplicates (2,673, counting the `NDUPLICATE` value), works-for-me (1,563), invalid (1,394) or won't-fix (1,267), among others. Descriptions have a median length of 413 characters, and 6.4% are longer than the 4,000-character limit applied to LLM inputs.

Both training and test sets are balanced. The study has no test set at natural prevalence; Section 5.3 estimates how the scores would shift under the pool's class mix, and Section 7.3 discusses what that estimate cannot cover.

4. Models and evaluation protocol

Figure 1 shows the study design and Figure 2 the two neural architectures that were trained. Each model family is described below with the settings recovered from the code and logs; Table 3 collects the training configurations of the learned models.

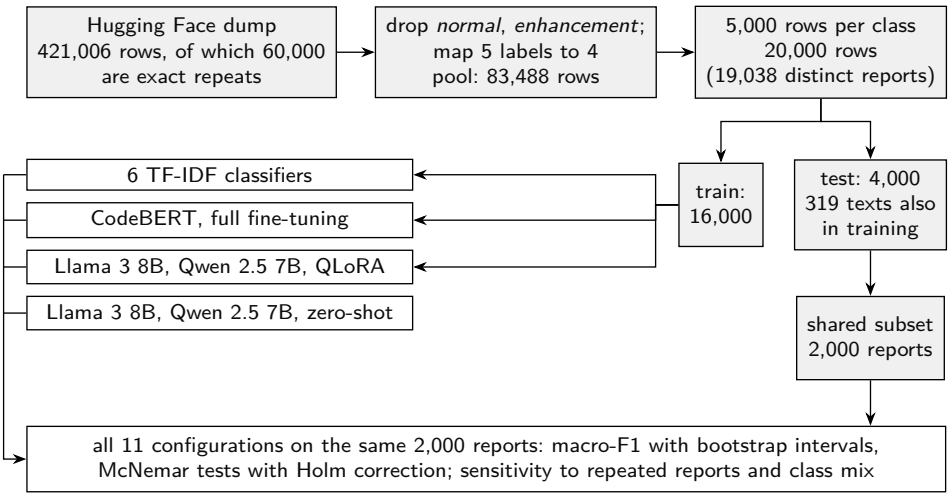


Figure 1: Study design. Grey boxes are data, with the counts found in the audit; white boxes are the eleven model configurations. The TF-IDF classifiers were fit on 12,800 of the 16,000 training reports. The shared subset is the 2,000-report sample on which the LLMs were run; every other model was re-scored on it from its stored files.

4.1. TF-IDF classifiers

Text was vectorised with TF-IDF over word 1–3-grams (20,000 features, sublinear term frequency, English stop words removed, minimum document frequency 5, maximum 0.75). Six scikit-learn classifiers (Pedregosa et al., 2011) were trained: multinomial Naive Bayes with uniform class priors, logistic regression, linear SVM, random forest, XGBoost, and a soft-voting ensemble of logistic regression, a calibrated linear SVM, random forest and Naive Bayes. Hyperparameters were chosen by 10-fold grid search on macro-F1 (selected values: C=1 for logistic regression, C=0.1 for the SVM, 500 trees and `min_samples_split`=5 for the forest, α=10 for Naive Bayes, 200 trees of depth 10 for XGBoost). Inside the ensemble the members keep fixed settings (C=1 for both linear models, 300 trees, α=0.1) and only the voting weights were searched, giving $(2, 2, 1, 1)$. Two details of the training script matter. The classifiers were fit on 12,800 of the 16,000 training reports; the other 3,200 served only as a printed check. And the vectoriser was fit once on those 12,800 reports before the grid search, so the cross-validation folds share its vocabulary and inverse document frequencies. This touches hyperparameter choice only; the test split was not involved.

4.2. CodeBERT

CodeBERT (Feng et al., 2020) (RoBERTa-base architecture, 124.6M parameters) was fine-tuned end to end with a four-way classification head using Hugging Face Transformers (Wolf et al., 2020). Inputs were truncated at 512 tokens, which cuts 13.2% of the test reports. Training ran for three epochs (Table 3). The training script evaluated on the test split after every epoch and kept the checkpoint with the best test macro-F1. The three per-epoch values were 0.478, 0.526 and 0.532, so the kept checkpoint is the final one and equals what a fixed three-epoch schedule would have produced; the test split was nonetheless consulted during training, and we flag it here rather than retrain.

4.3. Zero-shot LLMs

Llama 3 8B and Qwen 2.5 7B were queried through Ollama[2] with the default tags `llama3:8b` and `qwen2.5:7b`, which at the time of writing resolve to the instruction-tuned variants (4-bit Q4_0 and Q4_K_M builds respectively). The instruction in Listing 1 was wrapped in the Alpaca layout shown and sent to Ollama's `generate` endpoint, which places it in each model's chat template. Decoding used temperature 0.1 and at most 150 new tokens. No examples were included in the prompt and no output constraint was imposed; constrained and few-shot variants were not tested. The Ollama version and the model digests used

[2] `https://github.com/ollama/ollama`

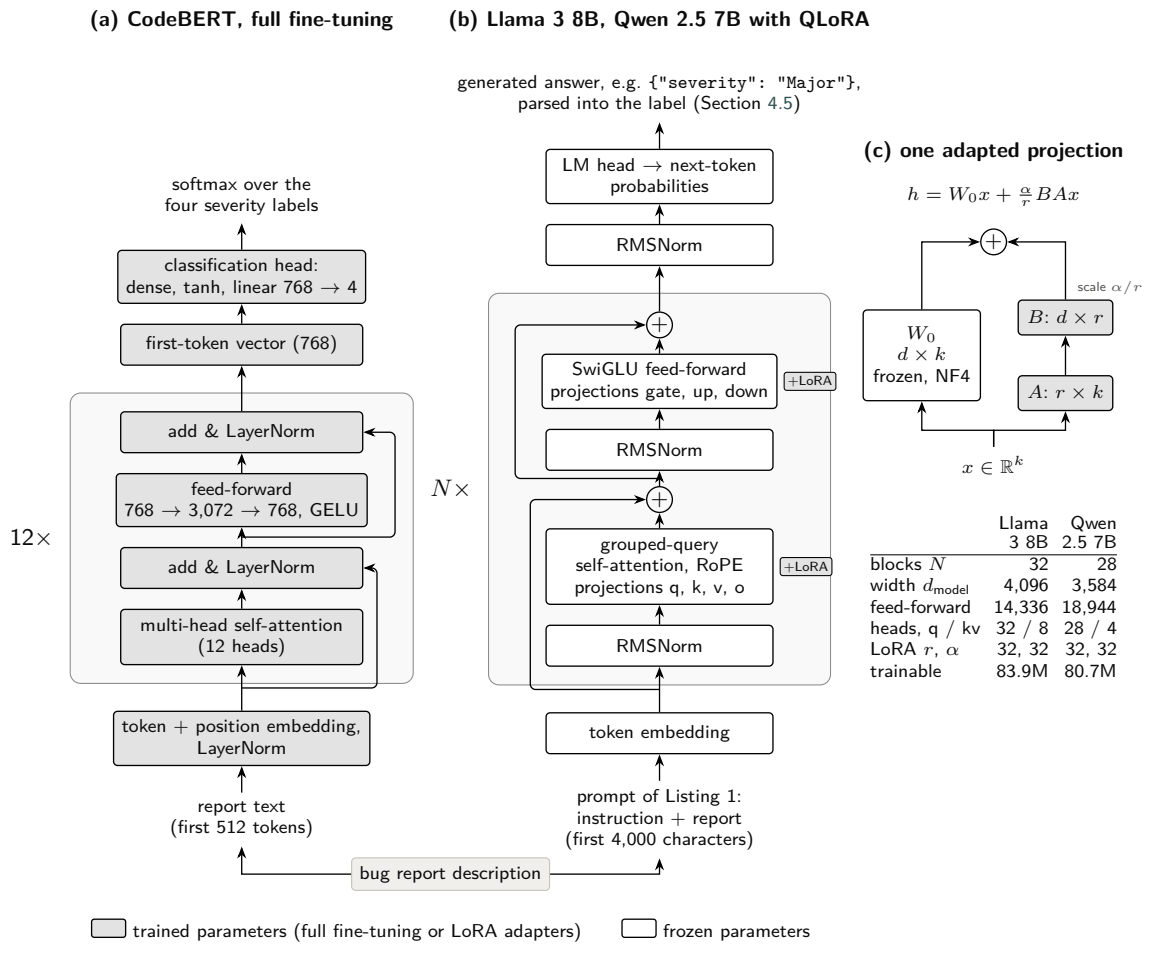


Figure 2: Architecture of the two trained neural classifiers. (a) CodeBERT, a 12-layer RoBERTa encoder fine-tuned end to end with a classification head. (b) The decoder-only LLMs, whose pre-trained weights stay frozen while LoRA adapters are trained on all seven linear projections of every block; the label is read from the generated text. During training the block projections are stored in NF4 with double quantisation, while the embeddings, normalisation layers and LM head stay in 16-bit. (c) One adapted projection (Eq. 1). Dimensions are taken from the CodeBERT configuration file and the headers of the deployed GGUF files, and trainable-parameter counts from the training logs.

at run time were not recorded, and the scripts did not set Ollama's context length, so the installed default applied.

4.4. QLoRA fine-tuning

Both LLMs were fine-tuned with Unsloth[3] on one NVIDIA A100 (40 GB) in Google Colab. Training started from the pre-trained base checkpoints, not the instruction-tuned ones, loaded in NF4. Each of the 16,000 training reports became one record in the format of Listing 1, so the fine-tuning prompt carries the same severity definitions as the zero-shot prompt. LoRA adapters (Hu et al., 2022) were attached to all seven linear projections of every transformer block (query, key, value and output in attention; gate, up and down in the feed-forward network). For a frozen matrix $W_0 \in \mathbb{R}^{d\times k}$ the adapted layer computes

$$h = W_0x + \frac{\alpha}{r}BAx, \qquad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times k}, \tag{1}$$

with $r = \alpha = 32$, so the scaling factor is one; during training, dropout with probability 0.05 is applied to the adapter input. Each adapted matrix adds $r(d + k)$ trainable parameters. For Llama 3 8B the seven

[3] https://github.com/unslothai/unsloth

```
Below is an instruction that describes a task, paired with an input that provides further context. Write a
     response that appropriately completes the request.

### Instruction:
You are a Senior QA Engineer. Classify the severity of the following bug report.

Severity definitions:
- Critical: System crash, data loss, security vulnerability.
- Major: Significant functionality is broken.
- Minor: Small functional issue or inconvenience.
- Trivial: Cosmetic issue or typo.

Output ONLY a valid JSON object with a single key:
{"severity": "<label>"}

### Input:
<bug report description, first 4,000 characters>

### Response:
{"severity": "Major"}<end-of-sequence>        (training records only)
```

Listing 1: Instruction and layout used for zero-shot prompting and, with the gold answer appended, for every fine-tuning record. The fine-tuned models receive the same text at inference and are expected to complete the response.

projections in one block contribute $32 \times 81{,}920 = 2{,}621{,}440$ parameters, and the 32 blocks give the 83,886,080 reported in the training log, 1.03% of the adapted model. For a single 4096×4096 projection the adapter is $dk/\big(r(d+k)\big) = 64$ times smaller than the matrix. That ratio describes what is stored and updated, not compute or memory: the forward and backward passes still run through the whole frozen network.

Training minimised the next-token cross-entropy computed from the output logits, averaged over all predicted tokens of an effective batch $\mathcal{B}$ of eight records,

$$\mathcal{L}(\theta) = -\frac{\sum_{i \in \mathcal{B}} \sum_{t=2}^{T_i} \log p_\theta(x_{i,t} \mid x_{i,<t})}{\sum_{i \in \mathcal{B}} (T_i - 1)}, \tag{2}$$

where $x_{i,1}, \ldots, x_{i,T_i}$ are the tokens of record i: instruction, report, answer and the end-of-sequence token. The first token is never predicted; for Llama 3 it is the beginning-of-sequence token, which the Qwen 2.5 base tokenizer does not add. The script did not mask the prompt tokens, so the loss covers the report text as well as the answer, which is only 6 or 7 tokens long under either tokenizer. All settings are listed in Table 3. No validation split was carved out and no hyperparameter search was logged. Besides the two runs in Table 3, the repository holds the log of one more Llama 3 run, trained with the same LoRA settings on the unquantised 16-bit base model (paged 32-bit AdamW, 2 h 22 min) and exported as a 16-bit GGUF file; it was never evaluated and no predictions exist for it. We cannot rule out that earlier, unrecorded runs informed the settings. Peak GPU memory was not logged.

After training, the adapters were merged into 16-bit weights, converted to GGUF with llama.cpp[4] and quantised to Q4_K_M, a different 4-bit scheme from the NF4 used in training. All reported results come from these exported files, served with llama-cpp-python (context 4,096 tokens, at most 32 new tokens, temperature 0.1, repetition penalty 1.15) on an Apple M1 Pro laptop. The hardware comes from the first author's records; the logs do not name it. We did not compare predictions before and after export, so the cost of the export step itself is unknown.

4.5. Evaluation protocol

Shared test reports. Local LLM inference was slow, so the evaluation scripts drew 2,000 of the 4,000 test reports with NumPy's generator seeded at 42 and ran the four LLM configurations on that subset. We regenerated the sampling indices with the same call, checked that every stored LLM prediction matches its

[4] https://github.com/ggml-org/llama.cpp

Table 3
Training configuration of the learned models, as recorded in the training scripts and logs. LLM losses are averaged over each epoch and cover all tokens of each record; CodeBERT's log keeps no per-epoch average. The NF4 settings come from the quantisation configuration of the base checkpoints.

	CodeBERT	Llama 3 8B (QLoRA)	Qwen 2.5 7B (QLoRA)
Initial checkpoint	`microsoft/codebert-base`	`unsloth/llama-3-8b-bnb-4bit`	`unsloth/qwen2.5-7b-bnb-4bit`
Trainable / total parameters	124,648,708 / 124,648,708	83,886,080 / 8,114,147,328	80,740,352 / 7,696,356,864
Adaptation	full fine-tuning, classification head	LoRA $r = 32$, $\alpha = 32$, dropout 0.05, on q, k, v, o, gate, up, down	
Precision in training	fp32 weights, fp16 mixed precision	block projections frozen in NF4 with double quantisation (bitsandbytes); embeddings, norms and LM head in bf16; bf16 compute	
Objective	cross-entropy on 4 labels	next-token cross-entropy on the whole sequence (prompt not masked)	
Optimiser, learning rate	AdamW, 2×10^{-5}, linear decay, no warm-up	paged_adamw_8bit, 1×10^{-4}, cosine, 10% warm-up	
Weight decay	0.01	0.01	
Batch, epochs, steps	16, 3, 3,000	8 (2×4 accumulation), 3, 6,000	
Input limit	512 tokens	4,000 characters; 4096-token context	
Seed	42	3407	
Hardware, software	Colab GPU (type not logged)	A100-SXM4-40GB; Unsloth 2026.3.3, Transformers 5.2.0, PyTorch 2.10.0+cu128	
Training wall clock	0:23:28	3:01:12	2:39:47
Mean loss, epochs 1 / 2 / 3	not logged per epoch	1.091 / 0.915 / 0.760	1.054 / 0.916 / 0.795
Deployed artefact	PyTorch weights (fp32)	LoRA merged into 16-bit weights, GGUF Q4_K_M (llama.cpp)	

test row in text and gold label, and re-scored the six TF-IDF classifiers and CodeBERT from their saved model files on the same 2,000 reports. The subset contains 495 Critical, 505 Major, 491 Minor and 509 Trivial reports. Re-scoring the non-LLM models on all 4,000 reports reproduces the logged metrics to the fourth decimal (Appendix A).

Parsing. Zero-shot and fine-tuned outputs went through the same parser. It tries to read the whole response, and then the span from its first `{` to its last `}`, as a JSON object, and accepts the `severity` value if it equals one of the four labels, ignoring case. Otherwise it strips punctuation from the whole response and checks for the label names in the fixed order Critical, Major, Minor, Trivial, taking the first one that occurs anywhere. Only when no label name occurs, as in an empty response, a failed request or an answer such as "Blocker" or "High", does it record Minor. Every report counts in the metrics; none is dropped. The raw responses were not stored, so we cannot report how often each step fired or how many responses broke the requested format. The zero-shot errors therefore mix wrong labels with unusable output, and some Minor predictions may come from the default.

Metrics. With TP_c, FP_c and FN_c the per-class counts, precision and recall are $P_c = TP_c/(TP_c + FP_c)$ and $R_c = TP_c/(TP_c + FN_c)$, and the primary metric is

$$\text{macro-F}_1 = \frac{1}{4} \sum_c \frac{2P_c R_c}{P_c + R_c}. \tag{3}$$

We also report accuracy, macro-averaged precision and recall, per-class precision and recall, and confusion matrices.

Uncertainty and tests. Intervals come from a percentile bootstrap over reports (Efron and Tibshirani, 1993): we resampled the 2,000 reports with replacement 10,000 times and took the 2.5th and 97.5th percentiles of each statistic. The same resamples are used for every model, so intervals for between-model differences are paired. For ten comparisons selected from the research questions (Table 5) we report a two-sided bootstrap p-value for the macro-F1 difference, taken as twice the smaller share of resampled differences

Table 4
Results on the 2,000 shared test reports (495 Critical, 505 Major, 491 Minor, 509 Trivial). Acc. is accuracy; brackets give 95% bootstrap intervals. The last column (dedup.) repeats macro-F1 after removing the 159 shared reports whose text also appears in the training set ($n = 1{,}841$). Best macro-F1 in bold.

Family	Model	Acc.	Macro-P	Macro-R	Macro-F1 [95% CI]	F1, dedup.
TF-IDF	Naive Bayes	0.435	0.456	0.437	0.430 [0.409, 0.452]	0.425
	Logistic regression	0.491	0.488	0.490	0.489 [0.467, 0.510]	0.474
	Linear SVM	0.490	0.484	0.489	0.484 [0.462, 0.505]	0.471
	Random forest	0.491	0.486	0.491	0.483 [0.461, 0.504]	0.455
	XGBoost	0.479	0.475	0.479	0.476 [0.454, 0.498]	0.445
	Soft-voting ensemble	0.486	0.482	0.486	0.479 [0.457, 0.501]	0.458
Encoder	CodeBERT	0.518	0.522	0.517	0.518 [0.497, 0.539]	0.508
Zero-shot	Llama 3 8B	0.367	0.531	0.370	0.305 [0.286, 0.325]	0.302
	Qwen 2.5 7B	0.374	0.535	0.377	0.321 [0.300, 0.342]	0.320
Fine-tuned	Llama 3 8B	0.527	0.570	0.526	**0.526** [0.504, 0.548]	0.524
	Qwen 2.5 7B	0.518	0.510	0.517	0.485 [0.463, 0.506]	0.478

on either side of zero (an approximation obtained by inverting the interval), and an exact McNemar test of accuracy on the reports that exactly one of the two models gets right (McNemar, 1947; Dietterich, 1998). Each family of ten p-values is corrected with Holm's procedure (Holm, 1979). We call a difference detectable when its 95% interval excludes zero and its adjusted p-value is below 0.05. Reports are treated as independent, although the shared subset contains 18 repeated descriptions, and each model was trained once, so seed-to-seed variation is not part of these intervals.

5. Results

5.1. RQ1: fine-tuned LLMs against TF-IDF and CodeBERT

Table 4 and Figure 3 give the scores on the 2,000 shared reports. They fall into three groups. The zero-shot LLMs sit at 0.305 and 0.321, well below all six TF-IDF classifiers. Those classifiers span 0.430 to 0.489, with Naive Bayes clearly last and the other five within 0.013 of each other; on these 2,000 reports their intervals overlap almost completely. CodeBERT (0.518) and fine-tuned Llama 3 (0.526) form the top group, with fine-tuned Qwen 2.5 (0.485) level with the better TF-IDF models.

Table 5 tests the selected comparisons. Fine-tuned Llama 3 and CodeBERT show no detectable difference: the macro-F1 difference is +0.007 with an interval from −0.017 to +0.032, and of the 654 reports on which exactly one of them is right, Llama gets 336 and CodeBERT 318. Fine-tuned Qwen 2.5 is 0.033 below CodeBERT on macro-F1 (adjusted $p = 0.031$), yet the two have the same accuracy (333 against 332 discordant reports). The same split appears between the two LLMs: Llama 3 leads Qwen 2.5 by 0.041 in macro-F1 while their accuracies do not differ. Macro-F1 separates these models because it weights each class's F1 equally, and Qwen's errors are concentrated in one class (Section 5.2). Against the soft-voting ensemble, fine-tuned Llama 3 is ahead on both tests (macro-F1 +0.047, accuracy +0.041), CodeBERT is ahead on macro-F1 (+0.039) but not on accuracy after Holm correction ($p = 0.079$), and fine-tuned Qwen 2.5 shows no detectable difference from it.

For RQ1, then: on these reports fine-tuned Llama 3 8B shows no detectable difference from fine-tuned CodeBERT and is ahead of the TF-IDF ensemble, while fine-tuned Qwen 2.5 7B is below CodeBERT on macro-F1 and shows no detectable difference from the ensemble. Neither LLM is ahead of the encoder.

5.2. RQ2: zero-shot versus fine-tuned, per class

The fine-tuned runs score 0.221 (Llama 3) and 0.164 (Qwen 2.5) higher in macro-F1 than the zero-shot runs (Table 5). Part of that gap may come from the setup and not from training: the zero-shot runs used instruction-tuned checkpoints inside their chat templates, served by Ollama with a 150-token cap, while the

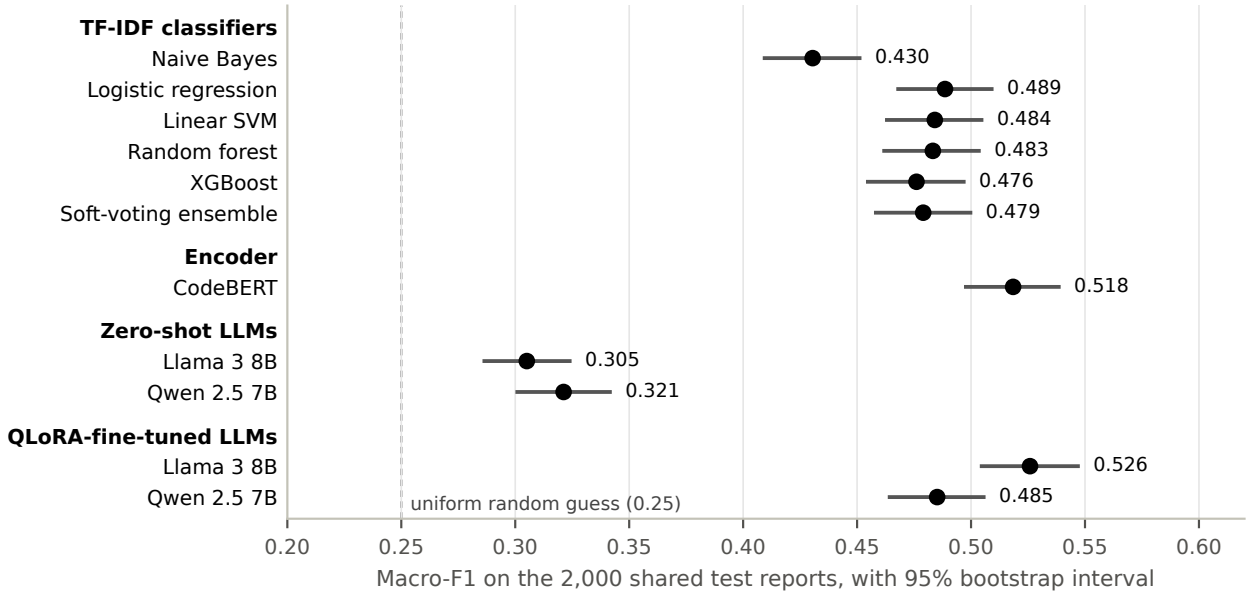


Figure 3: Macro-F1 of all eleven configurations on the shared 2,000 reports, with 95% bootstrap intervals.

Table 5
Selected pairwise comparisons on the shared 2,000 reports. FT is fine-tuned, ZS zero-shot, and the TF-IDF ensemble is the soft-voting ensemble. Δ is A minus B. b and c count reports that only A or only B classifies correctly. Bootstrap p-values cannot fall below $1/10{,}000$, hence the floor after Holm correction.

Comparison (A vs. B)	Δ macro-F1 [95% CI]	p_{Holm}, bootstrap	b / c	p_{Holm}, McNemar
Llama 3 FT vs. CodeBERT	$+0.007$ $[-0.017, +0.032]$	1.000	336 / 318	1.000
Qwen 2.5 FT vs. CodeBERT	-0.033 $[-0.058, -0.008]$	0.031	333 / 332	1.000
Llama 3 FT vs. Qwen 2.5 FT	$+0.041$ $[+0.015, +0.066]$	0.010	380 / 363	1.000
Llama 3 FT vs. Llama 3 ZS	$+0.221$ $[+0.195, +0.247]$	$\leq$0.001	581 / 262	$<$0.001
Qwen 2.5 FT vs. Qwen 2.5 ZS	$+0.164$ $[+0.133, +0.194]$	$\leq$0.001	748 / 459	$<$0.001
CodeBERT vs. TF-IDF ensemble	$+0.039$ $[+0.014, +0.065]$	0.010	373 / 309	0.079
Llama 3 FT vs. TF-IDF ensemble	$+0.047$ $[+0.020, +0.074]$	0.002	448 / 366	0.027
Qwen 2.5 FT vs. TF-IDF ensemble	$+0.006$ $[-0.021, +0.033]$	1.000	384 / 319	0.079
Llama 3 ZS vs. TF-IDF ensemble	-0.174 $[-0.201, -0.147]$	$\leq$0.001	385 / 622	$<$0.001
Qwen 2.5 ZS vs. TF-IDF ensemble	-0.158 $[-0.186, -0.130]$	$\leq$0.001	367 / 591	$<$0.001

fine-tuned runs used base checkpoints on raw text, served by llama-cpp-python with a 32-token cap and a repetition penalty; and zero-shot answers without a label name became Minor. Section 7.1 returns to this.

The per-class picture (Table 6, Figure 4) shows how the two conditions differ. Both zero-shot runs pull reports towards Minor, possibly in part through the parser default: Llama 3 predicts Minor for 1,276 of the 2,000 reports and Qwen 2.5 for 1,353, whereas Trivial is predicted 31 and 90 times. When they do say Critical they are usually right (precision 0.741 and 0.700), but they say it rarely, so Critical recall is 0.162 and 0.127.

After fine-tuning the two models settle into opposite trade-offs. Llama 3 leans towards Major, which it predicts 866 times. It finds 177 of the 495 Critical reports (recall 0.358, interval 0.316–0.400) and labels 293 of the rest Major, so 64% of Critical reports are missed, most of them one level down. Qwen 2.5 leans towards the two ends of the scale. It finds 69% of Critical reports (recall 0.691, interval 0.651–0.731) and 83% of Trivial ones, at the price of calling 242 Major reports Critical and 279 of the 491 Minor reports Trivial, leaving Minor recall at 0.214. CodeBERT is more even, with recall between 0.440 and 0.648 for every class.

Table 6
Per-class precision (P) and recall (R) on the shared 2,000 reports, with a 95% bootstrap interval for Critical recall.

	Critical		Major		Minor		Trivial		Critical recall
Model	P	R	P	R	P	R	P	R	95% CI
Soft-voting ensemble	0.518	0.513	0.447	0.311	0.408	0.458	0.553	0.660	[0.471, 0.557]
CodeBERT	0.557	0.471	0.423	0.440	0.454	0.511	0.655	0.648	[0.427, 0.515]
Llama 3 8B (zero-shot)	0.741	0.162	0.388	0.450	0.319	0.829	0.677	0.041	[0.129, 0.195]
Qwen 2.5 7B (zero-shot)	0.700	0.127	0.396	0.366	0.322	0.886	0.722	0.128	[0.099, 0.157]
Llama 3 8B (fine-tuned)	0.632	0.358	0.412	0.707	0.517	0.493	0.720	0.546	[0.316, 0.400]
Qwen 2.5 7B (fine-tuned)	0.524	0.691	0.484	0.335	0.495	0.214	0.536	0.827	[0.651, 0.731]

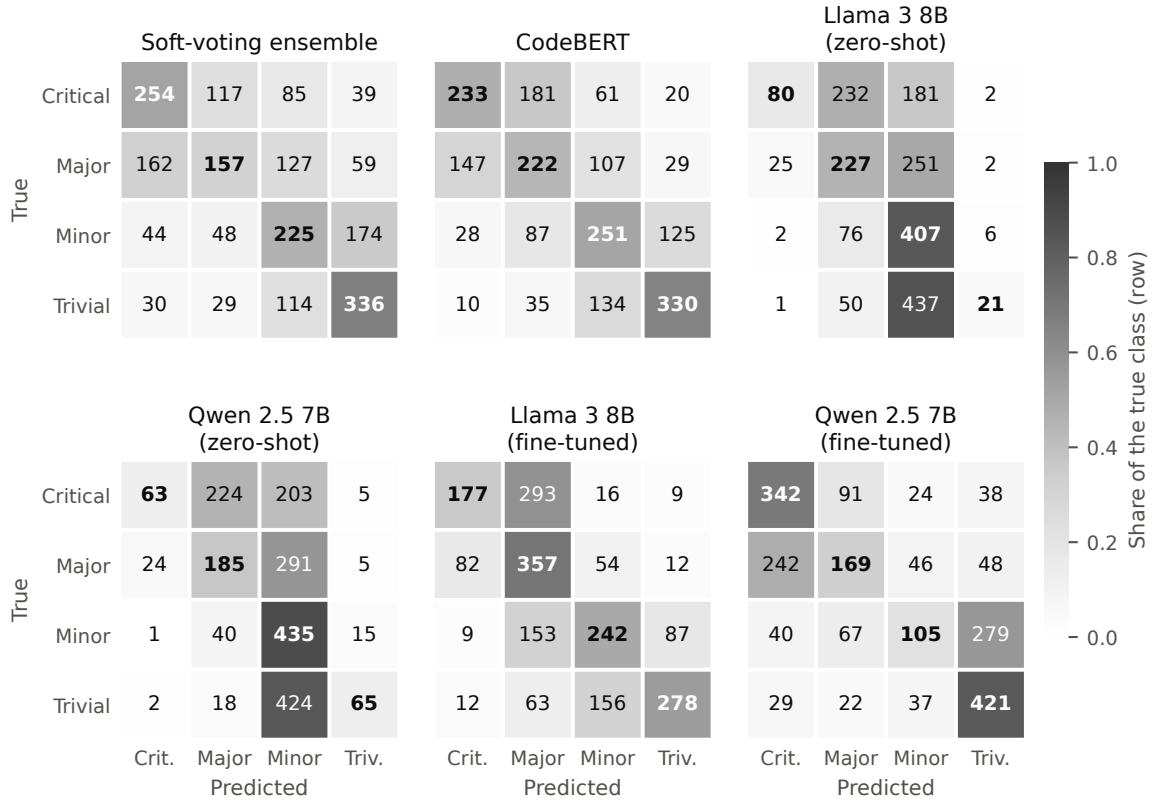


Figure 4: Confusion matrices on the shared 2,000 reports for six configurations. Cells show counts; shading shows the share of each true class (row). Rows are true labels, columns predicted labels.

Most errors of every model fall between neighbouring levels: 70–73% for the TF-IDF classifiers, 81% for CodeBERT, 79% for fine-tuned Qwen 2.5 and 87% for fine-tuned Llama 3. For the soft-voting ensemble the largest confusions are Minor with Trivial (288 reports) and Critical with Major (279), not Major with Minor (175). In short, for RQ2, the fine-tuned runs use the whole scale where the zero-shot runs concentrate on Minor, but the two fine-tuned models distribute their errors very differently, and neither finds Critical reports reliably.

5.3. RQ3: duplicates and class prevalence

Of the 2,000 shared reports, 159 have a description that also appears in the training set. Dropping them (last column of Table 4) lowers the TF-IDF scores by 0.005 to 0.031, most for XGBoost and random forest, and CodeBERT's from 0.518 to 0.508. The fine-tuned LLMs move by 0.007 or less, and the zero-shot runs,

Table 7
Inference wall-clock time on the Apple M1 Pro, from the most recent logged evaluation runs. The LLM loops also include prompt building and parsing; in the CodeBERT and LLM loops the first call also loads the model.

Model	Reports	Total (s)	Per report	Timed loop covers
TF-IDF classifiers (six)	4,000	0.39–1.09	0.10–0.27 ms	load model, vectorise, predict
CodeBERT	4,000	196.3	49 ms	per-report tokenise and forward pass
Llama 3 8B (zero-shot)	2,000	2,742	1.37 s	HTTP call to local Ollama, 150-token cap, 50 ms pause
Qwen 2.5 7B (zero-shot)	2,000	3,076	1.54 s	HTTP call to local Ollama, 150-token cap, 50 ms pause
Llama 3 8B (fine-tuned)	2,000	2,619	1.31 s	llama-cpp-python call, 32-token cap
Qwen 2.5 7B (fine-tuned)	2,000	2,693	1.35 s	llama-cpp-python call, 32-token cap

which never saw the training set, by 0.003 or less. The ordering behind RQ1 is unchanged on the remaining 1,841 reports: fine-tuned Llama 3 (0.524) and CodeBERT (0.508) are 0.016 apart, and both stay above every TF-IDF classifier (at most 0.474).

The small change for the fine-tuned LLMs does not show that they handle duplicates better. On the 159 shared reports whose text and label appear in training, random forest and XGBoost are right 86% and 87% of the time, CodeBERT 68%, fine-tuned Qwen 2.5 66% and fine-tuned Llama 3 56%; on the other shared reports the same five models score 46%, 44%, 50%, 51% and 52%. The gain on seen reports is 40 and 43 points for the two tree models, 18 for CodeBERT, 15 for Qwen 2.5 and only 4 for Llama 3. With 159 reports each of these rates is uncertain by up to about ±8 points, and only 126 of the 159 were in the 12,800 reports the TF-IDF models were fit on. Fine-tuned Qwen 2.5 therefore behaves much like CodeBERT, while fine-tuned Llama 3 gains little from having seen a report. Three explanations are possible for Llama 3 and we cannot separate them: limited fitting after three epochs of LoRA at a learning rate of 10^{-4}, the training loss, in which the 6 or 7 answer tokens are a small part of each record, and the export from NF4 training to a Q4_K_M file. Since 175 of the 319 repeated test reports are Trivial, the effect of duplication falls mostly on that class.

The test set is balanced, but the pool is not. As an estimate, not a measurement, we re-weighted each model's confusion matrix to the pool's class mix (Major 48.6%, Critical 26.3%, Minor 18.5%, Trivial 6.6%), assuming each model keeps its per-class behaviour. Under that mix the estimated macro-F1 falls by 0.046 to 0.078 for the TF-IDF classifiers, by 0.056 for CodeBERT (to 0.462) and by 0.080 for fine-tuned Qwen 2.5 (to 0.405), but by only 0.009 for fine-tuned Llama 3 (to 0.517). The reason is Trivial precision. With Trivial reports scarce, models that predict Trivial generously produce mostly false alarms: Trivial precision drops to 0.242 for Qwen 2.5 and 0.333 for CodeBERT, against 0.424 for Llama 3, whose errors go to the now common Major class. Under this estimate fine-tuned Llama 3 leads CodeBERT by 0.055 (95% bootstrap interval 0.029 to 0.081), but the interval does not cover the re-weighting assumption itself. For RQ3, then, duplication inflates the TF-IDF scores somewhat and leaves the LLM comparison intact, while the balanced test set may hide an advantage of fine-tuned Llama 3 over CodeBERT under a Major-heavy stream; only a test set drawn at natural prevalence can confirm or reject it.

5.4. Inference and training cost

Table 7 lists the wall-clock times logged by the evaluation scripts on the M1 Pro. CodeBERT needs 49 ms per report and fine-tuned Llama 3 1.31 s, about 27 times longer, with no detectable difference in macro-F1 on balanced data. The fine-tuned runs were 4.5% (Llama) and 12.5% (Qwen) faster than the zero-shot ones. We do not credit this to fine-tuning: the loops differ in serving runtime and output cap, and the zero-shot loop also pauses 50 ms per request, which adds 100 s per run (3.6% of the Llama zero-shot time). Training took 23 minutes for CodeBERT and three hours (Llama 3) and 2 hours 40 minutes (Qwen 2.5) for QLoRA on the A100 (Table 3).

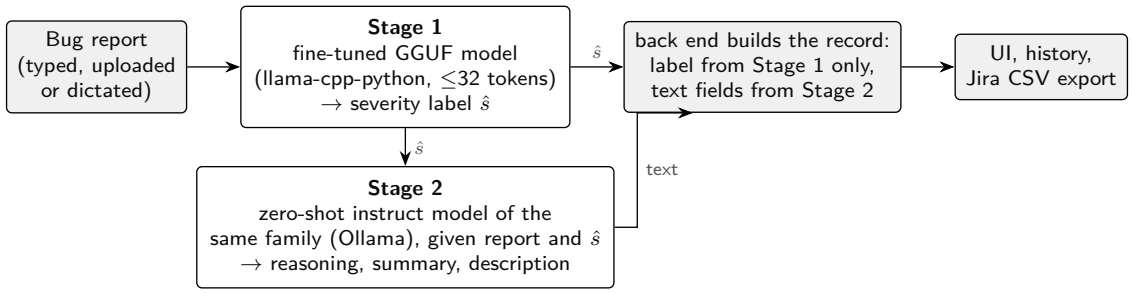


Figure 5: Two-stage analysis in CyberTriage. Any severity the Stage 2 model writes is discarded by the back end; the stored label is always Stage 1's.

6. CyberTriage: serving the classifier

The fine-tuned models are packaged in CyberTriage, an open web application with a React front end and a FastAPI back end, released with the analysis code at `https://github.com/tbayan/bug_finetuning_llm_artifacts`. As in our earlier web-based search platform for professional facilitator networks (Bayan, Nurbekova, Galy and Baigusheva, 2025a), the models are reached through a browser, so practitioners can use them without running scripts. A registry lets the user pick any of the evaluated models. Reports can be typed, uploaded as PDF, DOCX, TXT, LOG or JSON files, or dictated through the browser's speech-recognition interface. Analysed reports are kept in a history view and can be exported as a Jira import file, with Critical, Major, Minor and Trivial mapped to the Jira priorities Highest, High, Low and Lowest.

For the fine-tuned models, a single analysis runs in two stages (Figure 5). Stage 1 is the fine-tuned classifier evaluated above and produces only the label. Stage 2 sends the report and that label to the zero-shot instruction model of the same family. Its prompt states the label, tells the model not to disagree with it, and asks for a technical justification, a one-line summary and a short description as JSON (temperature 0.3, up to 1,024 tokens). The back end copies only those three fields from the Stage 2 response; a severity field in that response, if present, is ignored. The guarantee is therefore narrow and enforced in code, not by the prompt or the interface: Stage 2 cannot change the stored label. It is not a guarantee of correctness. The pipeline's label is Stage 1's label, which is wrong for 47% (Llama 3) and 48% (Qwen 2.5) of the shared test reports, and the Stage 2 text is a post-hoc rationale written by a different model from the one that decided. We expect it to justify a wrong label as readily as a right one, but we have not evaluated the explanations, and we did not time Stage 2, so we give no end-to-end latency for the two-stage path.

7. Discussion

7.1. What fine-tuning bought, and what it did not

The gap between the zero-shot and fine-tuned runs is large, but it should not be read as the value of task learning alone. Training was not the only thing that changed (Section 5.2): checkpoint, prompt template, serving stack, quantisation format and decoding settings changed with it. With the raw responses gone we also cannot separate format failures from wrong answers, so the Minor bias of the zero-shot runs may partly reflect the parser default. A clean test of what fine-tuning adds would run the same checkpoint under four conditions (plain zero-shot, zero-shot with output constrained to the four labels, constrained few-shot, and fine-tuned with the same constraint) and report output validity separately. We have not done this.

What the data do show is that fine-tuning on 16,000 labelled reports did not take an 8B model detectably beyond a fully fine-tuned 125M encoder on this test set; the interval allows a difference of up to 0.032 in favour of Llama 3 or 0.017 in favour of CodeBERT. On the balanced test set the practical case for the LLM is therefore weak: it costs far more per report on the same laptop (Table 7) and three hours of A100 time to train. Our prevalence estimate is the one result pointing the other way, and it depends on an assumption we could not test.

The two fine-tuned models also disagree with each other more than their aggregate scores suggest. Under identical data, prompt and hyperparameters they ended with opposite biases, one towards Major and one

towards the ends of the scale. We trained each model once and cannot say how much of this would survive another seed, so we do not attribute it to the architectures.

7.2. Implications for triage

For triage the per-class numbers matter more than the aggregate. Fine-tuned Llama 3 misses 64% of Critical reports and Qwen 2.5 misses 31%; CodeBERT misses 53%. With Critical recall between 0.36 and 0.69, no model here should route reports without a person checking the result, least of all where a missed Critical report has a real cost. The models are more useful as a second opinion than as a gate. Because most misses land one level down, reviewing every report predicted Major or above catches most Critical reports, at the cost of a long queue: on our balanced set this covers 95% of Critical reports for fine-tuned Llama 3 but means reviewing 57% of all reports, against 84% and 47% for CodeBERT. At the pool's class mix, where Major is common, the queue would grow to about 74% and 62% of reports. With score-based models one could instead lower the decision threshold for Critical and choose it on held-out data; we did not evaluate this.

7.3. Threats to validity

We follow the usual four categories (Wohlin, Runeson, Höst, Ohlsson, Regnell and Wesslén, 2012).

Construct validity. The ground truth is the severity recorded in Bugzilla, which is noisy (Tian et al., 2016); 41 identical descriptions in our pool carry conflicting labels, and we did not audit a sample by hand. Merging *blocker* into Critical and dropping *normal* define a narrower task than real triage, where *normal* is the most common label. Macro-F1 on a balanced set rewards equal performance across classes, which is not the same as the cost of errors in practice.

Internal validity. The zero-shot and fine-tuned conditions differ in more than training (Section 7.1). The LLM outputs were parsed with a fallback whose firing rate is unknown. CodeBERT's checkpoint was chosen on the test split, although the choice coincided with the last epoch. The TF-IDF classifiers saw 12,800 training reports and the other models 16,000. Each model was trained once with a single seed, and LLM inference sampled at temperature 0.1 without a fixed seed, so run-to-run variation is not covered by our intervals. We did not compare the NF4 training-time models with their Q4_K_M exports, and the Ollama version, model digests and context setting of the zero-shot runs were not recorded.

External validity. All data come from one project, Eclipse, and from reports that were later resolved; about 1% of descriptions include notes added after filing, and labels may have been revised after filing. The test set is balanced, the sample includes repeated reports, and we have neither a natural-prevalence test set nor a temporal or cross-project split. The prevalence estimate in Section 5.3 is a re-weighting and ignores the *normal* reports that would dominate a real stream.

Conclusion validity. The LLMs were evaluated on 2,000 of the 4,000 test reports, which widens intervals; differences below about 0.03 in macro-F1 are not resolvable here. The bootstrap treats reports as independent, although 41 test rows repeat a bug ID already in the test split (18 repeated descriptions in the shared subset). We limited formal testing to ten comparisons selected from the research questions and corrected for multiplicity within each test.

8. Conclusion

We compared TF-IDF classifiers, a fine-tuned CodeBERT encoder and two open 7–8B LLMs, prompted and QLoRA-fine-tuned, on the same 2,000 Eclipse bug reports, using numbers rebuilt from stored predictions. The zero-shot runs scored below every TF-IDF classifier. The fine-tuned runs scored far higher, in a setup that changed more than training. Fine-tuned Llama 3 8B showed no detectable difference from CodeBERT, at a much higher cost per report, and fine-tuned Qwen 2.5 7B was below CodeBERT on macro-F1. The fine-tuned models differed most in how they traded one class against another, and none of them found Critical reports reliably enough to work without review. The data audit showed that the public dump is Eclipse-only and repeats rows, and that the repeats reach the test set and inflate the TF-IDF scores while leaving the LLM comparison as it was.

The next steps follow from the gaps listed above: a natural-prevalence test set and a temporal split, deduplication before sampling, a validation split for every tuning decision, several seeds per model, a general-purpose encoder such as RoBERTa or DeBERTa as a second baseline, and the four-condition comparison

of prompting and fine-tuning with constrained output. The split identifiers, per-report predictions and the script that produces every table and figure in this paper are released so that these extensions can start from the same records.

A. Re-scored baselines on the full test split

Table 8 gives the TF-IDF classifiers and CodeBERT on all 4,000 test reports, re-scored from their saved model files, next to the macro-F1 and wall-clock time of the most recent logged evaluation run. The re-scored values match the logged ones. Scores on the full split are 0.002 to 0.016 higher than on the 2,000-report subset, so the subset is a slightly harder draw for these models.

Table 8
TF-IDF classifiers and CodeBERT on the full 4,000-report test split.

Model	Accuracy	Macro-P	Macro-R	Macro-F1	Logged F1	Logged time (s)
Naive Bayes	0.450	0.472	0.450	0.444	0.4439	0.52
Logistic regression	0.493	0.489	0.493	0.491	0.4908	0.39
Linear SVM	0.495	0.488	0.495	0.489	0.4887	0.60
Random forest	0.501	0.495	0.501	0.494	0.4939	1.09
XGBoost	0.495	0.490	0.495	0.492	0.4917	0.58
Soft-voting ensemble	0.498	0.492	0.498	0.490	0.4904	1.02
CodeBERT	0.531	0.535	0.531	0.532	0.5322	196.27

CRediT authorship contribution statement

Asset Kanatov: Conceptualization, Methodology, Software, Investigation, Data curation, Writing – original draft. **Talgar Bayan:** Supervision, Validation, Formal analysis, Writing – review & editing. **Askar Boranbayev:** Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare no competing interests.

Declaration of generative AI and AI-assisted technologies in the writing process

During the development of the CyberTriage software the first author used AI coding assistants (GitHub Copilot and Cursor). During the preparation of this manuscript the authors used Claude (Anthropic) to help re-run the verification analysis and to edit the text. The authors reviewed and edited all content and take full responsibility for the publication.

Data availability

The source data are public at `https://huggingface.co/datasets/AndressaStefany/bug-reports`. The repository `https://github.com/tbayan/bug_finetuning_llm_artifacts` holds the code, the training notebooks with their logs, the logged evaluation metrics, the split identifiers (bug ID, split, label, membership in the shared subset, duplicate flag), the per-report predicted labels of all eleven configurations, and the analysis script, which rebuilds the split from the source data and recomputes every table and figure from these files. The trained model files are not redistributed; every reported number can be recomputed from the released per-report predictions. The raw LLM responses were not stored.

Acknowledgements

This work builds on the first author's Master's thesis at Nazarbayev University (2026).

References

Agarwal, A., Madhavan, M., 2026. Fine-tuning code models for bug severity prediction: A comparative study with GraphCodeBERT and LoRA, in: Data Science and Applications. Springer Nature Switzerland. Lecture Notes in Networks and Systems, pp. 387–398. doi:10.1007/978-3-032-12827-0_30.

Ali, A., Khan, I.A., Xia, Y., Tian, Y., Sajid, S., Alsuhaibani, M., 2025. A retrieval-augmented LLM framework for severity prediction of bug reports in cloud-based mobile applications. Journal of Cloud Computing 14, 77. doi:10.1186/s13677-025-00826-w.

Ali, A., Xia, Y., Umer, Q., Osman, M., 2024. BERT based severity prediction of bug reports for the maintenance of mobile applications. Journal of Systems and Software 208, 111898. doi:10.1016/j.jss.2023.111898.

Arshad, M.A., Riaz, A., Fatima, R., Yasin, A., 2024. SevPredict: Exploring the potential of large language models in software maintenance. AI 5, 2739–2760. doi:10.3390/ai5040132.

Bayan, T., Banach, R., 2023. Exploring the privacy concerns in permissionless blockchain networks and potential solutions, in: 2023 IEEE International Conference on Smart Information Systems and Technologies (SIST), pp. 567–572. doi:10.1109/SIST58284.2023.10223536.

Bayan, T., Mukhambetiyar, B., Boranbayev, A., Yazici, A., 2026a. Dataset of round-level federated learning and layer-2 blockchain overhead from ten Raspberry Pi edge clients. Data in Brief 68, 113111. doi:10.1016/j.dib.2026.113111.

Bayan, T., Mukhambetiyar, B., Boranbayev, A., Yazici, A., 2026b. New dataset of performance metrics, device telemetry, and blockchain audit logs from a federated learning testbed on Raspberry Pi devices for HAR. Mendeley Data, version 2. doi:10.17632/dgmjr3ngtv.2.

Bayan, T., Nurbekova, Z., Galy, M.M., Baigusheva, K., 2025a. Enhanced multimodal diagram search for professional facilitator networks: A web-based educational platform, in: 2025 10th International Conference on Computer Science and Engineering (UBMK), pp. 1252–1257. doi:10.1109/UBMK67458.2025.11206913.

Bayan, T., Yazici, A., Banach, R., 2025b. Permissionless blockchain recent trends, privacy concerns, potential solutions and secure development lifecycle. Future Internet 17, 547. doi:10.3390/fi17120547.

Dao, A.H., Yang, C.Z., 2021. Severity prediction for bug reports using multi-aspect features: A deep learning approach. Mathematics 9, 1644. doi:10.3390/math9141644.

Dettmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L., 2023. QLoRA: Efficient finetuning of quantized LLMs, in: Advances in Neural Information Processing Systems 36 (NeurIPS 2023). arXiv:2305.14314.

Dietterich, T.G., 1998. Approximate statistical tests for comparing supervised classification learning algorithms. Neural Computation 10, 1895–1923. doi:10.1162/089976698300017197.

Efron, B., Tibshirani, R.J., 1993. An Introduction to the Bootstrap. Chapman & Hall, New York.

Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., Zhou, M., 2020. CodeBERT: A pre-trained model for programming and natural languages, in: Findings of the Association for Computational Linguistics: EMNLP 2020, pp. 1536–1547. doi:10.18653/v1/2020.findings-emnlp.139.

Grattafiori, A., et al., 2024. The Llama 3 herd of models. arXiv preprint arXiv:2407.21783.

Holm, S., 1979. A simple sequentially rejective multiple test procedure. Scandinavian Journal of Statistics 6, 65–70.

Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., Wang, H., 2024. Large language models for software engineering: A systematic literature review. ACM Transactions on Software Engineering and Methodology 33, 1–79. doi:10.1145/3695988.

Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., 2022. LoRA: Low-rank adaptation of large language models, in: International Conference on Learning Representations (ICLR). arXiv:2106.09685.

Kiashemshaki, K., Khosravani, A., Hosseinpour, A., Akhavan, A., 2025. Automated bug triaging using instruction-tuned large language models. arXiv preprint arXiv:2508.21156.

Lamkanfi, A., Demeyer, S., Giger, E., Goethals, B., 2010. Predicting the severity of a reported bug, in: 2010 7th IEEE Working Conference on Mining Software Repositories (MSR 2010), pp. 1–10. doi:10.1109/MSR.2010.5463284.

Lamkanfi, A., Demeyer, S., Soetens, Q.D., Verdonck, T., 2011. Comparing mining algorithms for predicting the severity of a reported bug, in: 2011 15th European Conference on Software Maintenance and Reengineering, pp. 249–258. doi:10.1109/CSMR.2011.31.

McNemar, Q., 1947. Note on the sampling error of the difference between correlated proportions or percentages. Psychometrika 12, 153–157. doi:10.1007/BF02295996.

Menzies, T., Marcus, A., 2008. Automated severity assessment of software defect reports, in: 2008 IEEE International Conference on Software Maintenance, pp. 346–355. doi:10.1109/ICSM.2008.4658083.

Pedregosa, F., et al., 2011. Scikit-learn: Machine learning in Python. Journal of Machine Learning Research 12, 2825–2830.

Qwen Team, 2024. Qwen2.5 technical report. arXiv preprint arXiv:2412.15115.

Ramay, W.Y., Umer, Q., Yin, X.C., Zhu, C., Illahi, I., 2019. Deep neural network-based severity prediction of bug reports. IEEE Access 7, 46846–46857. doi:10.1109/ACCESS.2019.2909746.

Silva de Oliveira, A.S., 2024. Predictive Power of Large Language Models in Bug Severity Detection: An Explorative Study. Master's thesis. Polytechnique Montréal. URL: https://publications.polymtl.ca/58326/.

Tian, Y., Ali, N., Lo, D., Hassan, A.E., 2016. On the unreliability of bug severity data. Empirical Software Engineering 21, 2298–2323. doi:10.1007/s10664-015-9409-1.

Tian, Y., Lo, D., Sun, C., 2012. Information retrieval based nearest neighbor classification for fine-grained bug severity prediction, in: 2012 19th Working Conference on Reverse Engineering, pp. 215–224. doi:10.1109/WCRE.2012.31.

Wohlin, C., Runeson, P., Höst, M., Ohlsson, M.C., Regnell, B., Wesslén, A., 2012. Experimentation in Software Engineering. Springer, Berlin, Heidelberg. doi:10.1007/978-3-642-29044-2.

Wolf, T., et al., 2020. Transformers: State-of-the-art natural language processing, in: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pp. 38–45. doi:10.18653/v1/2020.emnlp-demos.6.

Yang, G., Zhang, T., Lee, B., 2014. Towards semi-automatic bug triage and severity prediction based on topic model and multi-feature of bug reports, in: 2014 IEEE 38th Annual Computer Software and Applications Conference, pp. 97–106. doi:10.1109/COMPSAC.2014.16.